Password Hash Cracking Report

Target: TryHackMe – “Crack the Hash” Room

Prepared by: Manoj Bora

Date: 14 September 2026

Classification: For Academic / Training Use Only

Document Control

Report Title	Password Hash Cracking – TryHackMe Crack the Hash
Target	TryHackMe room “Crack the Hash” (hashes provided directly within the room)
Tester	Manoj Bora
Engagement Type	Guided training lab (TryHackMe) – hash identification and cracking exercise
Date of Testing	14 September 2026
Date of Report	14 September 2026
Classification	For Academic / Training Use Only

Executive Summary

This report covers the TryHackMe “Crack the Hash” room, completed as part of my Advanced Diploma in Cyber Security coursework. The room presents nine password hashes across two difficulty levels and asks the tester to identify each hash type and recover the original plaintext password.

Using hashes.com for identification and lookup, I successfully recovered all nine passwords — five from unsalted, common hash formats in Level 1 (MD5, SHA1, SHA256, bcrypt, MD4), and four from salted or less common formats in Level 2 (SHA256, NTLM, SHA512crypt, and salted SHA1). The room finished at 100% completion. Full per-hash results and takeaways are covered in the Findings and Summary sections below.

Scope & Objectives

Target: TryHackMe room “Crack the Hash” (tryhackme.com/room/crackthehash) — hash values, and salts where applicable, were supplied directly within the room; no external host was involved.

Type of exercise: guided lab — all material needed to attempt each hash was provided by the room itself.

Objective: correctly identify each hash's algorithm and recover its original plaintext password, progressing from simple unsalted hashes to salted and less common formats.

Methodology

The same repeatable process was followed for each hash provided in the room:

- Hash Identification — inspecting the hash's format, length, and prefix to narrow down the likely algorithm.
- Lookup & Cracking — submitting the hash, along with its salt where one was provided, to hashes.com to retrieve a match.
- Verification — submitting the returned plaintext as the answer in TryHackMe and confirming a “Correct Answer” response.

1. Room Setup & Tools

No target host or network scanning was involved in this room — all material (hashes, and salts where relevant) was provided directly within the TryHackMe task pages. The main body of work was hash identification and lookup, tracked through to a 100% room completion.

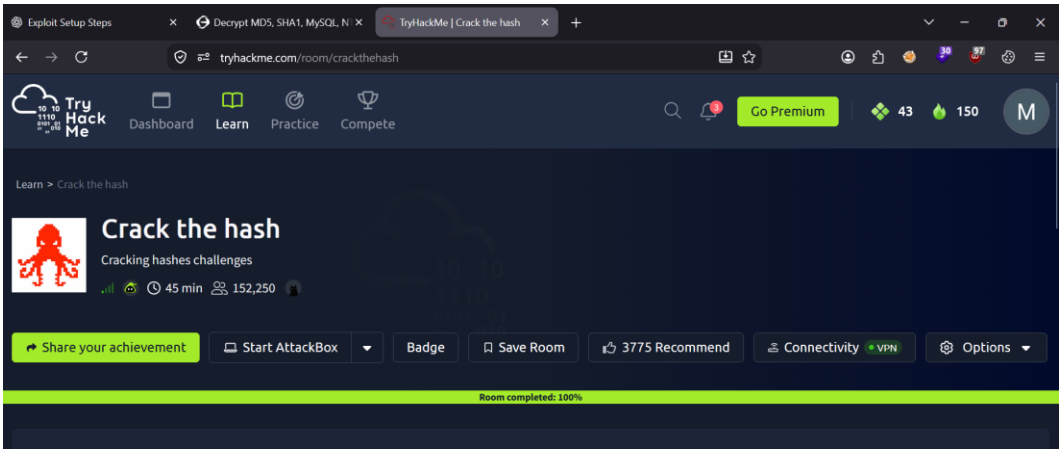


Figure 1 – Crack the Hash room completed at 100%

Tools Used

Tool	Type	Purpose
hashes.com	Online hash ID / cracking service	Identifying hash algorithms and retrieving cracked plaintext for every hash in the room
Web browser	Client	Accessing the TryHackMe room and hashes.com
TryHackMe platform	Lab environment	Hosting the room's hashes and salts, and validating submitted answers

Findings

The room's two tasks form a natural difficulty progression rather than an attack chain — Level 1 covers common, unsalted hashes, while Level 2 introduces salting and less common formats. Each is treated as a finding below, with full per-hash results in the accompanying tables.

Finding 1: Task 1 – Level 1 (Common, Unsalted Hashes)

What I did:

- Copied each hash from the TryHackMe task into hashes.com.
- Let hashes.com auto-identify the algorithm and return the matching plaintext.
- Submitted each recovered password back into TryHackMe and confirmed a “Correct Answer” result for all five hashes.

Hash	Type	Cracked Password
`48bb6e862e54f2a795ffc4e541caed4d`	MD5	`easy`
`CBFDAC6008F9CAB4083784CBD1874F76618D2A97`	SHA1	`password123`
`1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032`	SHA256	`letmein`
`$2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom`	bcrypt	`bleh`
`279412f945939ba78ce0758d3fd83daa`	MD4	`Eternity22`

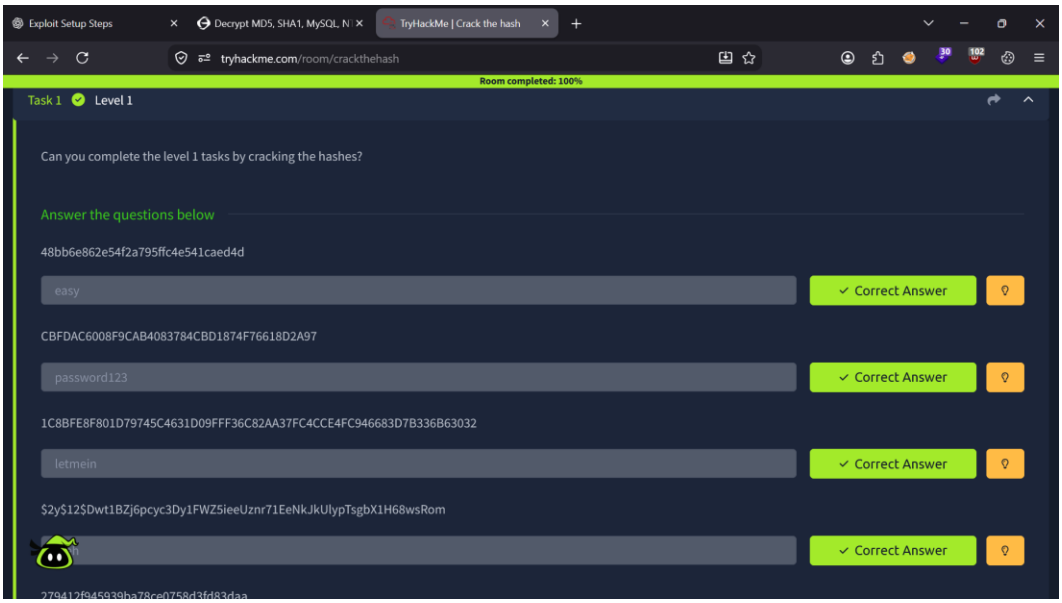


Figure 2 – Task 1 (Level 1) – all answers submitted correctly

Why it matters: All five hashes here use common, unsalted algorithms, and every plaintext ("easy", "password123", "letmein", "bleh", "Eternity22") was already present in public lookup databases. This shows how quickly a weak or previously-breached password can be recovered once its hash is exposed, regardless of which algorithm produced it.

Key takeaway: Algorithm choice matters less than password strength when a hash is unsalted and the password is common or reused elsewhere — unsalted hashes of weak passwords are effectively public information.

Finding 2: Task 2 – Level 2 (Salted & Less Common Hash Formats)

What I did:

- Identified each hash's type from its prefix (e.g. "6" for SHA512crypt) and length/structure for NTLM.
- Where the room supplied a salt (e.g. "aReallyHardSalt"), submitted the hash together with its salt to hashes.com rather than the hash alone.
- Retrieved and verified all four plaintexts, again confirming "Correct Answer" for each in TryHackMe.

Hash	Type	Cracked Password
`F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85`	SHA256	`paule`
`1DFECA0C002AE40B8619ECF94819CC1B`	NTLM	`n63umy8lkf4i`
`$6$aReallyHardSalt$6WKUTqzq...KZAs02.`	SHA512crypt (salted)	`waka99`
`e5d8870e5bdd26602cab8dbe07a942c8669e56d6`	SHA1 (salted)	`481616481616`

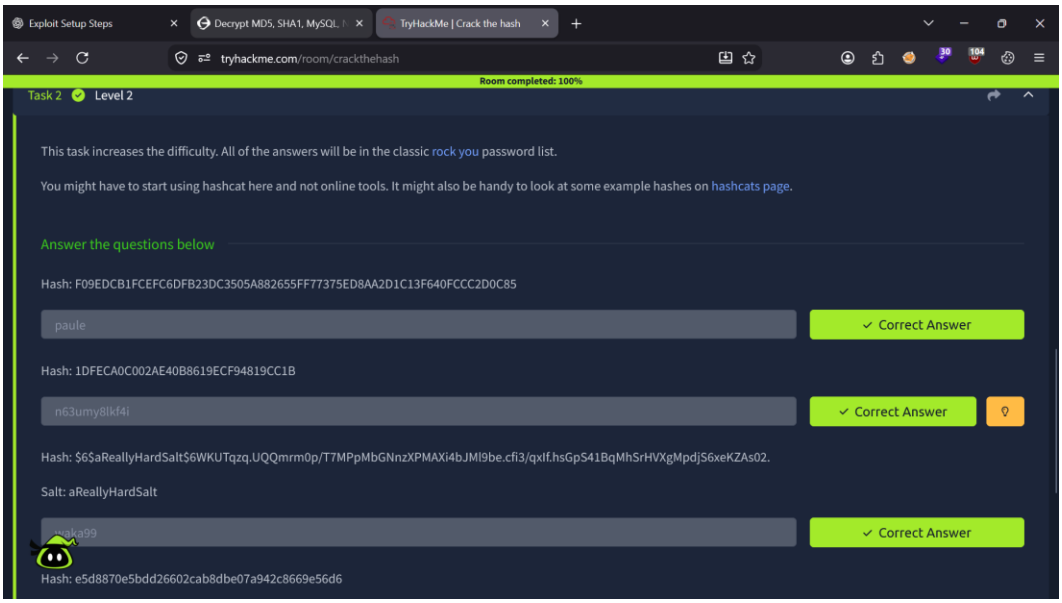


Figure 3 – Task 2 (Level 2) – salted hashes solved correctly

Why it matters: Salting is designed to defeat exactly the kind of precomputed lookup used in Finding 1 — the same password hashed with a different salt produces a completely different digest, so rainbow tables and shared lookup databases stop working. These hashes only became solvable because the room disclosed the salt alongside the hash, which mirrors how a real attacker would need to obtain both the hash and its salt (e.g. from a leaked /etc/shadow file) before an offline attack is even possible.

Key takeaway: A salt does not make a weak password strong — it only removes the shortcut of reusing someone else's precomputed crack. Once the salt is known, a weak or dictionary-based password is still recoverable; it just takes a dedicated lookup or an offline tool like Hashcat instead of a generic database hit.

Hash Strength / Crack Difficulty Summary

Hash Type	Typical Use Case	Offline Crack Difficulty*	Notes
MD5	Legacy password / checksum hashing	Low	Deprecated for passwords; near-instant lookup or brute force
SHA1	Legacy password hashing	Low	Deprecated; fast to brute force or match via lookup
SHA256	General-purpose hashing	Low (unsalted)	Fast per-guess, so unsalted use for passwords is still weak
MD4	Legacy / NTLM base algorithm	Low	Obsolete and trivial to crack
bcrypt	Modern password storage	High	Deliberately slow (cost factor); resists brute force even though it doesn't stop weak passwords
NTLM	Windows authentication	Low–Medium	Fast to compute; needs a wordlist/lookup match rather than a rainbow table
SHA512crypt	Unix/Linux password storage (/etc/shadow)	High	Salted and computationally expensive by design

** Difficulty ratings reflect offline cracking speed against a standard wordlist (e.g. rockyou.txt). Real-world resistance also depends heavily on password complexity, not just the algorithm used.*

Conclusion & Lessons Learned

The “Crack the Hash” room was completed at 100%, recovering all nine passwords across seven distinct hash algorithms using hashes.com. Level 1 demonstrated how quickly common, unsalted hashes fall to lookup databases, while Level 2 showed that salting — not just a “stronger” algorithm — is what actually breaks that shortcut, and that slow algorithms like bcrypt and SHA512crypt exist specifically to make brute forcing expensive even when a salt is known.

What I'd practice next:

- Re-crack the same hashes locally with Hashcat/John the Ripper against rockyou.txt, noting the mode numbers used (e.g. -m 3200 for bcrypt, -m 1800 for SHA512crypt) to build offline cracking skills rather than relying on an online lookup.
- Time each crack to compare how much slower bcrypt/SHA512crypt are than MD5/SHA1 in practice.
- Use dedicated identification tools (hashid, Hash Type Identifier, hashcat's example-hashes page) instead of relying solely on hashes.com to identify a hash.
- Follow up with a Hashcat-focused room or “Crack The Hash Level 2” to build a natural next entry in the portfolio.

Overall, this room reinforced that password security depends on the combination of hashing algorithm, salting, and password strength together — weakening any one of the three undermines the others.